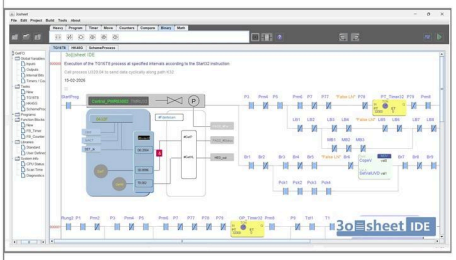
Programmable Machine Controller (PMC)

A proprietary, world-first hardware-software ecosystem powered by 3o|||sheet IDE

> **Unprecedented Architecture:** This is a proprietary, industry-first concept that has no direct analogies on the market. Unlike traditional automation setups that rely on separate PLC hardware and HMI panels communicating over slow protocols, the PMC completely merges the industrial controller, I/O modules, and operator controls (buttons, joysticks, encoders, and touchscreens) into a single, unified execution layer.

All-in-One CPU ModuleUnified core for both standalone **PLC execution** and **physical operator panels.**

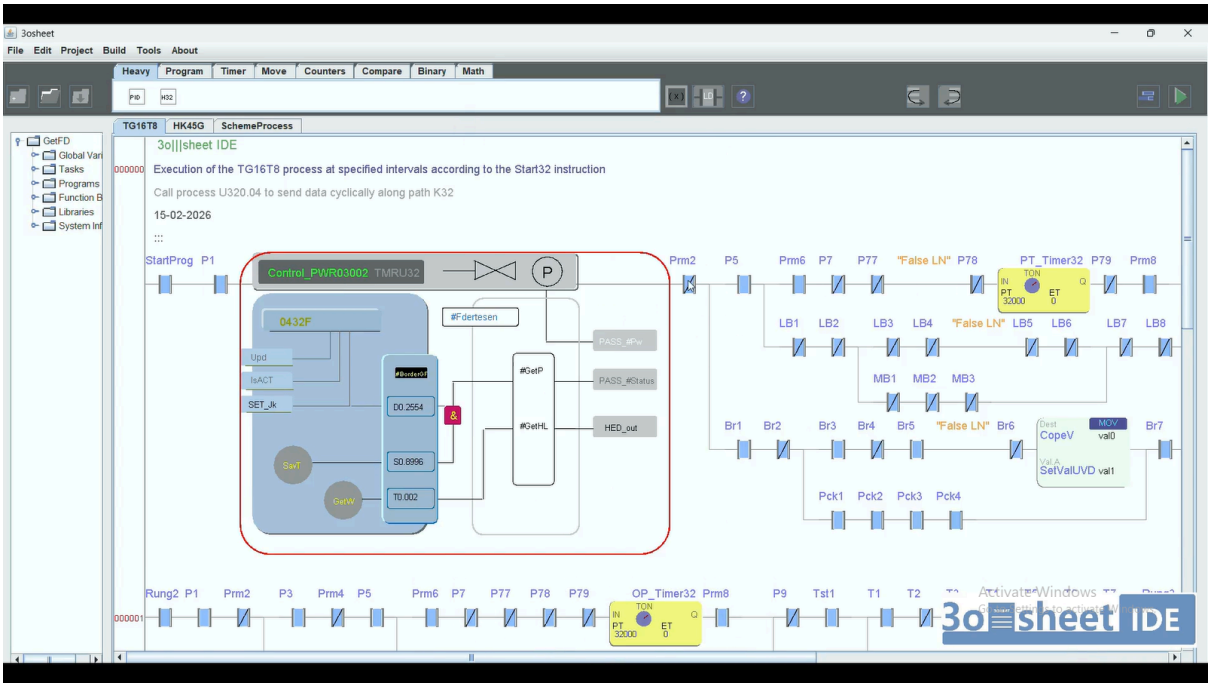


UNIFIED DEVELOPMENT ENVIRONMENT
3o|||sheet IDE:
One software for both PLCs and Control Panels.Hybrid Code Canvas: Seamlessly mix LD and FBD in a single rung.

Unified Core Architecture & Single Interface

Traditional industrial automation strictly separates control logic from the user interface. The PMC breaks that paradigm with a proprietary Unified Core architecture where a single computing core executes machine logic and directly drives all physical and touchscreen interface elements via a unified connection interface on a DIN rail or latch system.

- **All-in-One CPU Module:** A single proprietary controller handles standard industrial I/O processing, physical operator inputs, and touchscreen coordinates simultaneously.
- **Zero-Protocol Mapping:** Hardware inputs, analog joysticks, digital encoders, and virtual touchscreen buttons map directly to internal variables, eliminating communication latency and configuration overhead.



Inter-Module Interface & Bus Architecture (36 - 72 IO + CAN)

Within the **PMC (Programmable Machine Controller)** ecosystem, the connection between the CPU module and peripheral blocks utilizes a hybrid interface (socket-to-plug connector) consisting of two independent data-sharing layers:

1. **Parallel Direct I/O Bus (36-72 IO Lines):** A dedicated physical layer designed for instantaneous hardware-level connections.
2. **Serial CAN Bus:** A communication layer dedicated to module initialization, dispatching, and low-priority data transmission.

Dynamic Resource Allocation (Zero-Protocol Mapping)

When a physical operator panel module docks with the shared bus, the interaction follows a two-stage hybrid handshake:

1. **Registration & Interrogation (CAN Layer):**
    - The connected module broadcasts an initialization packet to the CPU via the **CAN** bus.
    - This packet contains the module's digital profile, specifying the type and count of its onboard controls (e.g., *buttons, sliders, encoders, joysticks*).
    - The CPU analyzes the current resource map and transmits back **assigned line numbers** allocated from the available 36 IO pool.
2. **Direct Hardware Commutation (Zero-Protocol Layer):**
    - Upon receiving its line assignments, the module's hardware routes its physical components (buttons, sliders) directly onto the designated lines of the 36-channel bus.
    - Signal transmission then bypasses traditional software protocol stacks entirely via **Zero-Protocol Mapping**. The physical controls map directly to internal controller variables at the hardware level.

Data Prioritization & System Scaling

The platform strictly segregates data streams based on their latency tolerance and critical priority:

- **High-Priority Elements (Zero-Protocol):**
  Reserved exclusively for physical operator interface controls (buttons, joysticks, encoders) that require instantaneous system response [pdf_YxndrK.pdf]. Direct hardware routing over the 36 IO lines completely eliminates delays caused by packet packaging, network arbitration, or software stack processing.
- **Standard-Priority Expansion (CAN Protocol):**
  Once all 36 direct I/O lines are fully allocated, any further system expansion shifts **exclusively to the CAN interface**.

Core Architectural Advantages

- **Zero Communication Latency:** Achieved for critical human-machine interface elements by keeping them on the hardware direct-wire layer
- **Dynamic Plug-and-Play:** Operator panels can be hot-swapped or rearranged on the fly; the system automatically re-routes physical lines within the bus.
- **CPU Resource Protection:** High-density, high-throughput standard I/O traffic is safely isolated within the CAN channel, preventing it from consuming ultra-fast direct-response lines.